DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Portfolio Project

CMSC 197 GDD Final Output

Prepared by: Rene Andre Bedonia Jocsing

Published: January 29, 2026 **Deadline:** May 21, 2026

The portfolio project is your semester-long original game, developed from concept to deployment. This document is **living**—it evolves with your game. What starts as a 1-page pitch in Week 2 becomes a comprehensive design document tracking your architecture, mechanics, and technical decisions by Week 16.

Portfolio Project Overview

- **Duration:** Entire semester (16 weeks)
- **Team Size:** Pairs (same as machine problems)
- **Weight:** 40% of final grade
- **Scope:** Original game, publishable quality
- **Repository:** GitHub with semantic commits throughout

Philosophy: The Living Document

Your concept document is not write-once-and-forget. It's a **living technical specification** that grows alongside your game. Professional studios maintain design documents that evolve through development—you'll do the same.

Modern game development increasingly integrates AI-assisted workflows, from code generation to asset creation. Living documents serve a dual purpose: they keep human teams aligned on vision and architecture, and they provide AI systems with structured context to prevent hallucinations and maintain consistency. This is why studios don't "wing it"—they plan, document, iterate, and refine with precision.

Why Living?

Game development is iterative. Your Week 2 vision will change by Week 8. Mechanics you thought were core might get cut. Systems you didn't anticipate become essential. The living document captures this evolution:

- **Week 2:** High-level pitch (1 page)
- **Week 5:** Expanded with core loop diagram, controls (2-3 pages)
- **Week 8:** Architecture sections added (3-4 pages)
- **Week 12:** Full systems documentation (5-7 pages)
- **Week 16:** Complete technical design doc (8-10 pages)

Common Pitfalls

This is NOT:

- A static essay you write once and submit
- A collection of vague ideas without technical detail
- A substitute for working code

This IS:

- A technical blueprint that guides implementation
- A communication tool between you and your partner
- A record of design decisions and their rationale

Document Structure by Milestone

Week 2: Living Concept Document (10%)

Length: 1 page

Purpose: Establish vision and feasibility

Required Sections:

1. **Title & Tagline**
   - Game name
   - One-sentence hook ("Roguelike dungeon crawler with Pokemon-style type effectiveness")
2. **Core Gameplay Loop**
   - What does the player DO? (3-5 bullet points)
   - Example: "Explore procedural dungeons, capture creatures, use type advantages in turn-based combat"
3. **Genre & Inspirations**
   - Primary genre (platformer, top-down shooter, etc.)
   - Reference games ("Binding of Isaac meets Pokemon")
4. **Technical Feasibility**
   - What systems do you need? (FSMs, pathfinding, inventory, etc.)
   - What's your biggest technical risk?
   - How will you validate feasibility by Week 5?
5. **Scope Management**
   - Minimum viable product (core loop only)
   - Stretch goals (if time permits)
   - What you're explicitly NOT building

Scope Management Strategy

Define your MVP as "the smallest version that demonstrates the core hook." If your hook is "type-based combat," MVP is one level with 3 creature types. NOT a full campaign with all 151 original Pokemon.

Good scope: Core mechanic + 1-2 levels + basic UI

Bad scope: Full campaign, achievements, multiplayer, modding support

Week 5: First Playable Prototype (20%)

Document Update: Expand to 2-3 pages

Deliverable: Playable build demonstrating core loop

New Sections to Add:

1. **Core Loop Diagram**
   - Visual flowchart of gameplay loop
   - State transitions (menu → gameplay → game over)

2. **Controls Schema**
   - Input mappings (WASD, mouse, etc.)
   - Rationale for control choices
3. **Prototype Learnings**
   - What worked in playtesting?
   - What's getting cut or reworked?
   - Technical surprises (good or bad)
4. **Updated Scope**
   - Revised MVP based on prototype
   - Adjusted timeline for remaining milestones

Prototype Requirements:

- Core mechanic functional (even if ugly)
- Player can win and lose
- 1-2 minutes of gameplay loop
- GitHub commit history shows iterative development

Week 8: Alpha Check-In (5%)

Document Update: Expand to 3-4 pages

Purpose: Catch architectural issues early

New Sections to Add:

1. **Architecture Overview**
   - Scene hierarchy diagram
   - Signal flow between major systems
   - Design patterns in use (State, Command, Observer, etc.)
2. **Technical Debt Log**
   - Known issues or hacks
   - Planned refactors
   - Why you made these tradeoffs
3. **Alpha Goals**
   - What features should work by Week 12?
   - What's still placeholder/programmer art?

Alpha Build Requirements:

- All core systems present (even if not polished)
- At least 5 minutes of gameplay
- No game-breaking bugs
- Architecture demonstrates scalability

Common Pitfalls for Alpha

Alpha is NOT feature-complete! Alpha means "all systems integrated, stability reasonable." Polish, balance, and content come later. If you're still prototyping systems at Week 8, you're behind schedule.

Week 12: Midterm Presentation (20%)

Document Update: Expand to 5-7 pages

Deliverable: 20-minute presentation + Q&A

New Sections to Add:

1. **Systems Documentation**
   - Enemy AI (FSMs, pathfinding)
   - Combat/interaction system

- Progression systems (leveling, unlocks, etc.)
- Save/load architecture (if applicable)

2. **Asset Pipeline**
   - Where assets come from (free, commissioned, self-made, AI-generated)
   - Asset credits and licenses
   - Placeholder vs. final art plan
3. **Playtesting Feedback**
   - What did playtesters struggle with?
   - What changes are you making based on feedback?
4. **Beta Roadmap**
   - What gets finished Weeks 13-15?
   - What's getting cut for time?
   - Polish priorities

Presentation Structure:

- 7 min: Live gameplay demo
- 7 min: Architecture deep-dive (show code/diagrams)
- 3 min: Challenges and solutions
- 3 min: Q&A

Week 15: Beta Check-In (5%)

Document Update: Minor additions (stay 5-7 pages)

Purpose: Feature lock, focus on polish

Sections to Update:

1. **Feature Lock Status**
   - Final feature list
   - What got cut and why
2. **Known Issues**
   - Bugs that won't be fixed by Week 16
   - Performance concerns
   - Edge cases
3. **Final Week Plan**
   - Bug fixing priorities
   - Polish tasks (juice, sound, UI)
   - Presentation preparation

Beta Build Requirements:

- Feature complete (no new systems)
- 10-15 minutes of gameplay
- Professional UI/HUD
- Export builds successfully
- Performance stable (no lag spikes)

Week 16: Final Presentation (40%)

Document Final: 8-10 pages

Deliverable: 20-minute presentation + polished game

Final Sections to Add:

1. **Postmortem**
   - What went well?
   - What would you change?
   - Key learnings
2. **Architecture Reflection**

- Which design patterns saved you?
- Where did your architecture break down?
- How would you architect it differently?

3. **Code Statistics**
   - Lines of code
   - Number of commits
   - Refactors performed
4. **Complete Credits**
   - All asset attributions
   - Third-party code/plugins
   - Playtester acknowledgments

Final Presentation Structure:

- 7 min: Full gameplay demo (start to finish)
- 5 min: Architecture showcase (best patterns/systems)
- 3 min: Design evolution (what changed and why)
- 3 min: Postmortem highlights
- 2 min: Q&A

Final Deliverable Checklist:

- Game builds for Windows/Linux/macOS (at least one)
- Complete living document (see format requirements below)
- GitHub repository with full history
- README.md with installation and controls
- Gameplay trailer (optional but recommended)

Presentation Schedule Note

On presentation dates (Weeks 12 and 16), class will begin at **8:00am sharp** to accommodate all pairs. Arriving late may result in reduced presentation time.

Document Formatting Guidelines

Required Format

- **File:** `GAME_NAME_DesignDoc` with appropriate extension
- **Location:** `/docs/` directory in repository
- **Versioning:** Commit after each milestone update
- **Acceptable Formats:**
  - Markdown (`.md`) + generated PDF
  - LaTeX (`.tex`) + compiled PDF
  - PDF only (if using Google Docs or similar)

All three formats (source + PDF) must be committed and updated incrementally throughout the semester. This shows your iterative process in version control.

Visual Requirements

- Use diagrams for architecture (scene trees, FSMs, signal flow)
- Include screenshots of gameplay at each milestone
- Embed code snippets for key systems (properly syntax-highlighted)
- Use clear section headers and table of contents

Diagramming Tools

Recommended tools for technical diagrams:

- **draw.io** (free, web-based, used in CMSC 127)
- **Mermaid** (text-based diagrams in Markdown)
- **Godot scene exports** (right-click scene tree, Copy Node Path)

Technical Writing Standards

- **Be specific:** "Player has 3 lives" not "Player has lives"
- **Show rationale:** Don't just say WHAT, explain WHY
- **Reference code:** Link to specific files/functions in repo
- **Update ruthlessly:** Strike through cut features, don't delete them

Professional Standards

Your living document is part of your portfolio. When applying for jobs, you'll show this to demonstrate:

- Technical communication skills
- Architectural thinking
- Ability to scope and deliver
- Process documentation discipline

Treat it as a professional artifact.

Documentation Examples

Good vs. Bad Technical Writing

Bad Example (Vague): "The player will have a combat system where they can fight enemies. The enemies will use AI to attack the player. There will be different types of weapons."

Good Example (Specific): "Combat uses a turn-based system where player and enemy alternate actions. Enemy AI is implemented via a 3-state FSM (Idle, Chase, Attack) with transitions based on distance thresholds (Chase at <300px, Attack at <100px). We chose FSM over behavior trees because our AI behaviors are mutually exclusive and simple. The player has 3 weapon types (Sword, Bow, Magic) each with different damage/range/cooldown tradeoffs (see `weapons.gd:15-42`)."

Bad Example (No Rationale): "We used the Observer pattern for the UI system."

Good Example (With Rationale): "We used Godot's signal-based Observer pattern for UI updates because it decouples the HUD from game logic. When the player's health changes, `player.gd` emits `health_changed(new_value)`, which the HUD listens for. This allows us to add/remove UI elements without touching player code, and enables future features like damage numbers or screen effects to subscribe to the same signal."

Common Mistakes to Avoid

Don't Do This:

- Writing document once and never updating it
- Vague descriptions ("There will be enemies")
- No diagrams or visual aids
- Describing features that don't exist in code

- Ignoring technical debt or known issues
- Treating it like a creative writing assignment

Document vs. Code Mismatch

Your document should reflect reality. If you describe a feature but it's not in the build, that's a red flag. Update the document when you cut features—strike through and add a note explaining why.

Overcomplicated Diagrams

Keep diagrams readable. A scene tree with 50 nodes is hard to parse. Show the major systems and key relationships, not every single node.

Missing the "Why"

Don't just document WHAT your game does. Explain WHY you made design decisions. "We used a Command pattern for input handling because it allows remappable controls and input playback for replays."

Late Submission Policy

Schedule Flexibility

Life happens. If you miss a milestone deadline:

- You may submit that deliverable up until the **next milestone**
- Late submissions receive a maximum grade of **60%** for that specific milestone
- This policy does **not apply** to the Week 12 and Week 16 presentations (no extensions possible)
- Both document update AND build must be submitted together

Example: Miss Week 8 alpha? Submit by Week 12, but alpha check-in capped at 3% (60% of 5%).

If you're falling behind, attend consultation hours immediately. Early intervention prevents catastrophic schedule slips.

Grading Per Milestone

Week 2 (10%) (February 6, 2026)

- Clear, specific core loop
- Realistic scope assessment
- Technical feasibility addressed

Week 5 (20%) (February 27, 2026)

- Playable prototype demonstrates core hook
- Document expanded with diagrams
- Scope updated based on prototype learnings

Week 8 (5%) (March 20, 2026)

- Architecture documented with diagrams
- Technical debt acknowledged
- Alpha build shows integrated systems

Week 12 (20%) (April 17, 2026)

- Strong presentation with clear demo

- Comprehensive systems documentation
- Evidence of playtesting and iteration

Week 15 (5%) (May 8, 2026)

- Feature lock documented
- Beta build is polished and stable
- Known issues cataloged honestly

Week 16 (40%) (May 15, 2026)

- Complete, polished game
- Professional final presentation
- Thoughtful postmortem
- Living document tells coherent story of development

Final Notes

The living document is not busywork. It's a communication tool that helps you and your partner stay aligned, a planning document that keeps you on track, and a portfolio piece that demonstrates your professionalism.

Studios don't build games by "winging it." They plan, document, iterate, and refine. You're learning that process here.

General Rubric (Applied to Each Milestone)

Note: Presentation criterion only applies to Weeks 12 (20%) and 16 (40%). For other milestones, weight redistributes proportionally across remaining criteria.

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Document Quality (30%)	Meets page target, specific technical detail with rationale, clear diagrams, proper versioning in repo	Meets page target, adequate detail, some diagrams, committed to repo	Under/over page target, vague descriptions, missing diagrams, inconsistent commits	Far from target length, no technical depth, no diagrams, not in repo
Build Quality (30%)	Meets all milestone requirements, stable, demonstrates architectural thinking	Meets most requirements, minor bugs, reasonable architecture	Meets minimum requirements, significant bugs, weak architecture	Missing core requirements, broken, no clear architecture
Scope Management (20%)	Realistic scope, clear MVP/stretch distinction, honest assessment of progress	Mostly realistic, some scope creep, reasonable progress tracking	Unrealistic scope or too conservative, poor progress tracking	Scope completely unmanageable or trivial, no progress awareness

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Individual Contribution* (10%)	Equitable commits with semantic messages, clear authorship, equal participation in presentations	Mostly balanced contributions, adequate commits, both members participate	Uneven contributions visible, poor commit messages, one member dominates	One member carries project, nonsemantic commits, minimal participation
Presentation (10%, Weeks 12 & 16 only)	Professional demo, articulate explanations, handles Q&A confidently, equal speaking time	Good demo, clear explanations, both members speak	Rough demo, unclear explanations, uneven participation	Unprepared, can't explain decisions, one member silent

**Score may vary individually for component.*